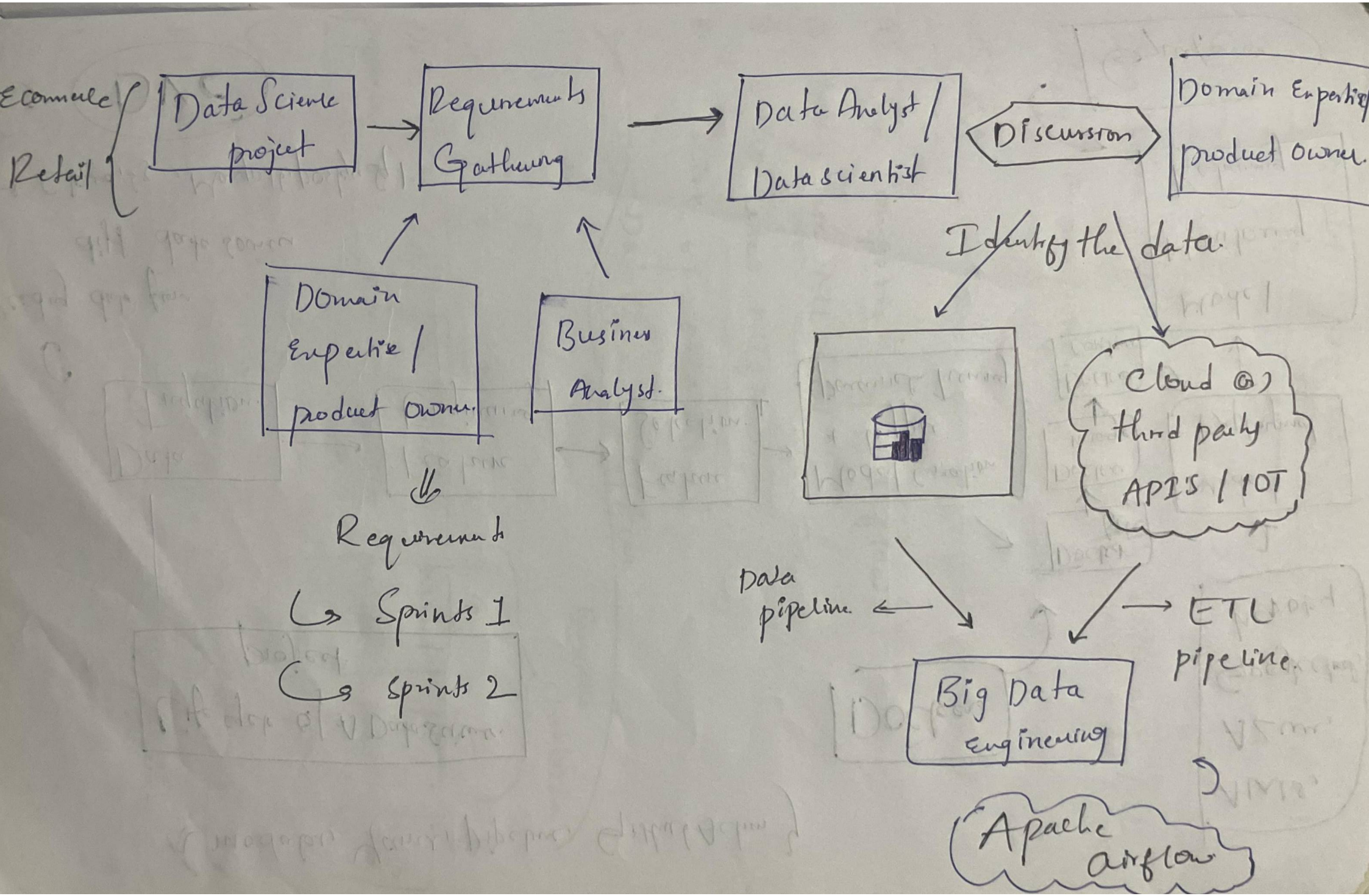
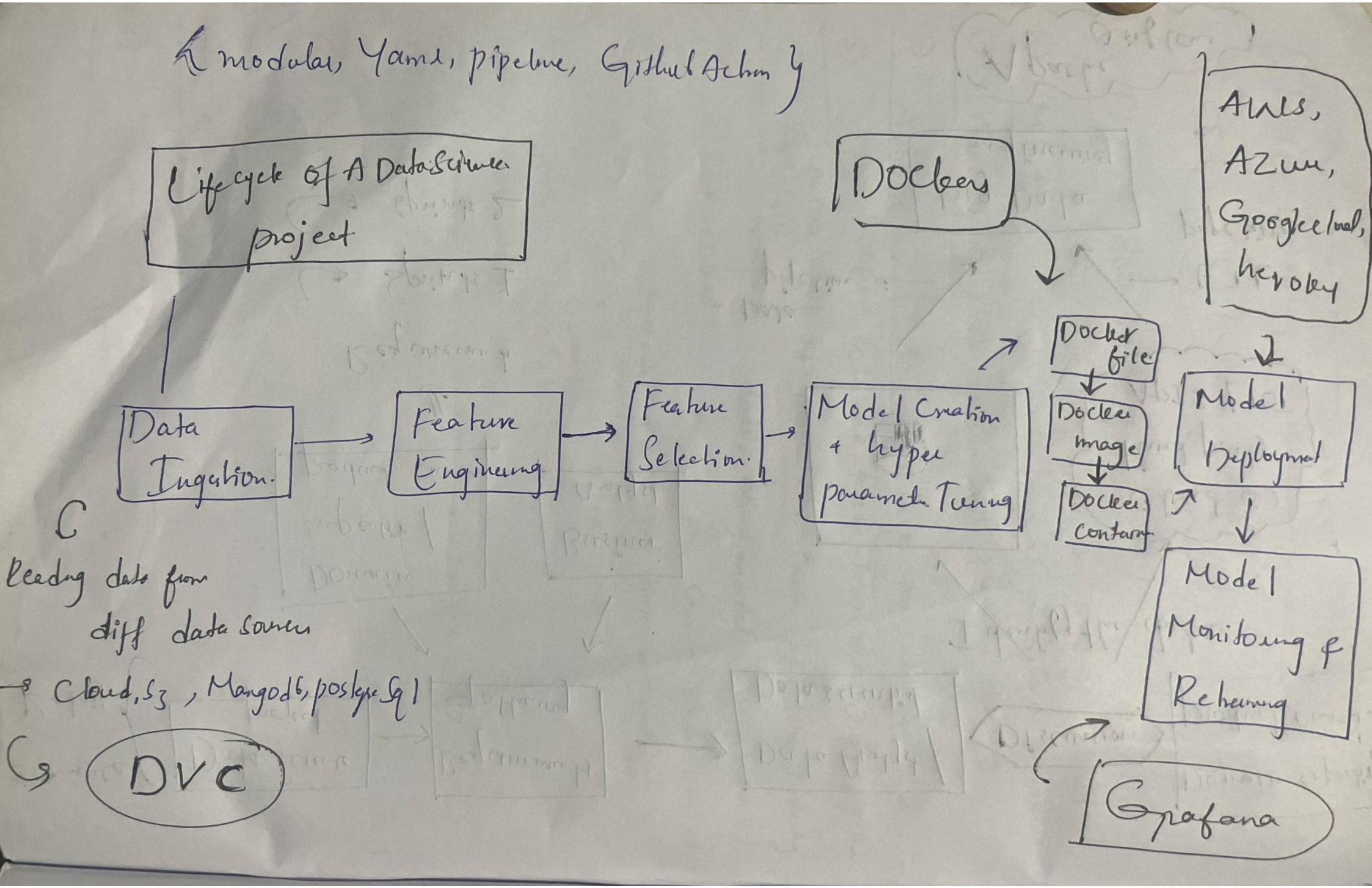
Tools Im going to learn in this Course:

1. DVC (data Versioning Control)
   - ↳ tool that helps you save and track different Versions of your data and models in machine learning.
2. MLflow
   - ↳ Experiments tracking.
3. Apache Airflow
4. Astronomer
   - ↳ tool that helps you automate and ↳ airflow schedule tasks in a ml (or) data pipeline
5. Git
   - ↳ code commit It make sure each step (like collecting
6. Github
   - ↳ repo data, training model, saving results)
7. AWS → cloud runs in the correct Order and at
8. Docker → docker use model the right time without manual
9. Github Actions work.
   - ↳ workflow
10. Amazon SageMaker.
11. Grafana → visualization (model performance)
12. MongoDB → DB
13. postgreSql → DB
14. python → programing language.
15. Azure → DB
16. Dagshub → Remote Repo.


Ecommerce
Retail
Data Science project
Requirements Gathering
Data Analyst / Data scientist
Discussion
Domain Expertise / product owner.
Identify the data.
Domain Expertise / product owner.
Business Analyst.
Requirements
Sprints 1
Sprints 2
Cloud @) third party APIS / IOT
Data pipeline
ETL pipeline.
Big Data engineering
Apache airflow

{ modular, Yaml, pipeline, Github Action }



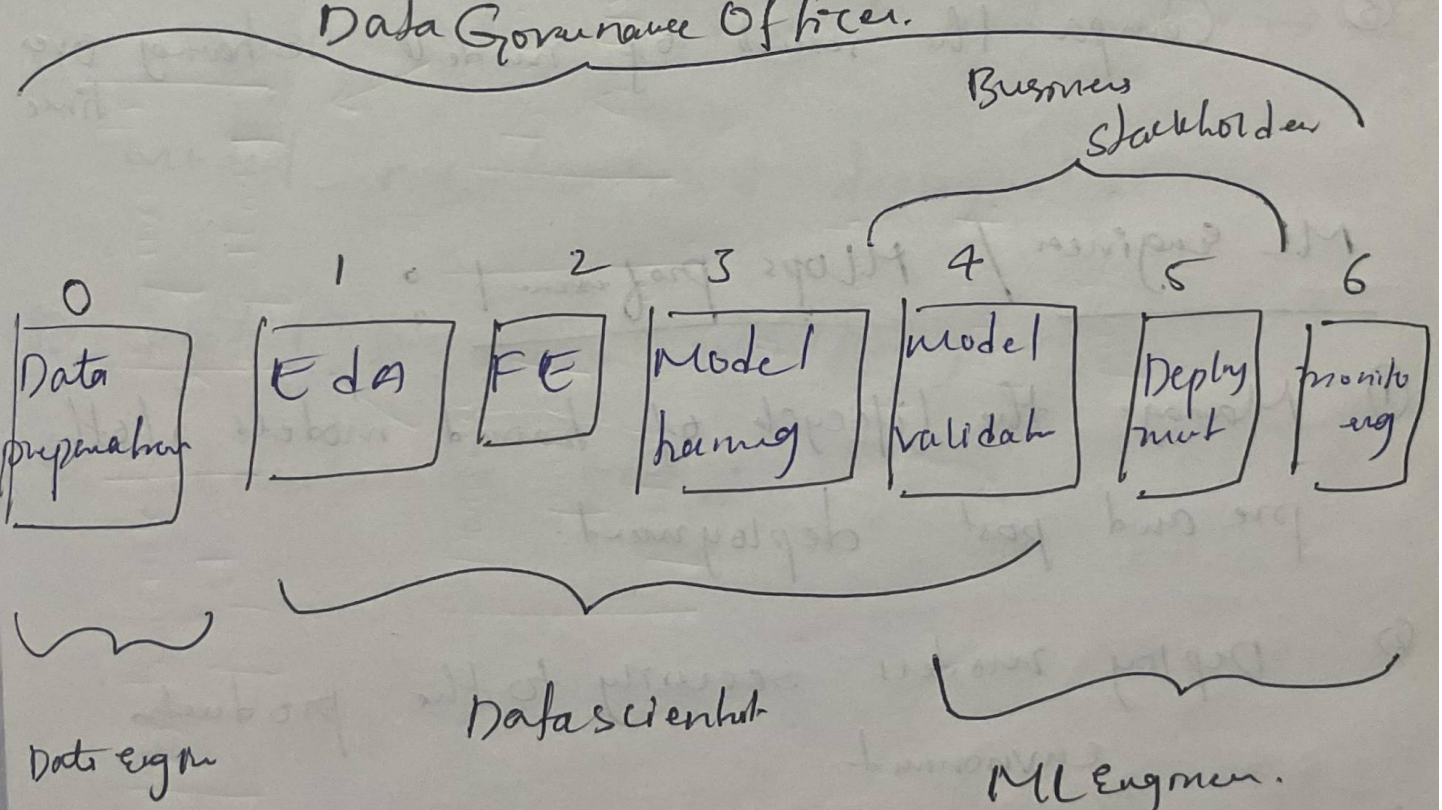
Complete MLflow

MLflow:

Open source tool that integrate with any ML library and platform.

Q&A

1. Core Components of MLflow
2. Why use MLFLOW
3. Who uses MLflow
4. Usecase of MLflow



Datascientist leveages MLFlow for:

1. Experiment tracking and hypothesis testing
2. Code structuring.
3. model packaging and dependency management
4. Evaluating hyperparameter. ***
   - Train model → parameter
   - tracks hyperparameter tuning
   - visualization Graphs
5. Compare the results of models achieving over time

ML engineer / MLops professional:

1. Manage the lifecycle of trained models, both pre and post deployment.
2. Deploy models securely to the production environment
3. Manage deployment dependencies.

prompt engineering users:

① Evaluate and experiment with large language models (LLM).

② Creating custom prompts and experiment

③ Deciding on the best base model suitable for project Requirement.

Uselases:

① Experiment Tracking

② Cs tracks parameters and metrics → UI

model selection and deployment

③ Model performance monitoring

④ Collaborative project.

MLFlow

Creating Mlflow Environment

- Create new folder
- Creating new environment
- Conda create -p venv python==3.10 -y
- Create requirements.txt
  - add mlflow
- pip install -r requirements.txt

Mlflow tracking Server

→ open terminal
- mlflow ui → runs on local server
  - ↳ import mlflow
  - ↳ Set tracking uri to local server i.e (http://127.0.0.1:5000)
  - ↳ Create Experiment
  - ↳ run mlflow

→ We can track Experiments through mlflow ui and while running mlflow, a folder (mlruns) created in the same directory, where we can also track Experiments.

*** If we stop running mlflow ui on server. then we are not going to track any experiments.

→ If we delete mlruns folder, there is no way to track mlflow ui in server.

First ML project with MLFlow

→ Check out in projects folder.

Inferencing model artifacts with Mlflow inferencing

- Inferencing and validating model through mlflow (artifacts)
  - Copying code from mlflow artifacts and validating.
- Loading the model back for prediction as a generic python function model.
  - load_model = mlflow.pyfunc.load_model (model_info.model_uri).

Model Registry: (mlflow model registry):

is a Centralized storage system where machine learning models are Saved, ~~versioned~~ versioned, tracked and managed. It helps in Organizing models, keeping track of different versions and smoothly moving models from development to production.

→ Initially we should not register the model, like how we done previously.

Why do we Register a model:

① To save and track all trained models

② to compare different versions and choose the best one later

③ to collaborate with teams, so others can review and test it.

④ To move models to production when they meet performance goals

practical use case of DVC in Vscode

On one large dataset, you could have made

10 changes

add changes to dvc ← dvc add dataset.csv ← dataset.csv

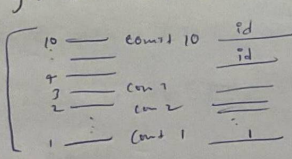
Track changes through git ← dataset.csv.dvc ← git add dataset.csv.dvc

↳ git commit -m "____"

Suppose you could have made 10 versions of it and .dvc folder generates 10 different .md5 files in cache folder. for accessing any of those we should follow this.

→ present git on master branch.

→ git log



getting backto recent one

→ git checkout master

→ dvc checkout

→ git checkout [commit you want to check]

↳ that points to that version .md5 file, but no change in data is seen

→ dvc checkout [that particular data is seen]

DVC (Data Version Control)

DVC is an open source version control system for machine learning projects that helps track data, models and experiments, similar to git but optimized for large files and datasets

→ Data and model versioning
→ Efficient storage and sharing } Versioning data
→ Reproducibility.

Dags Hub

is a remote repository for machine learning projects, similar to github, but designed for data, models and experiments using git, DVC and MLflow

three main features:

① Upload data to Dagshub (upload)
② Track your code (using git)
③ Log Experiments (MLflow)

→ cd dvc

→ git clone https://dagshub.com/Bala27/Newdemo.git

→ cd Newdemo dagshub

Beginner friendly DVC commands

① **Initialize DVC**

↳ dvc init

↳ git init

② **add data to dvc**

dvc add data.csv

↳ dvc files start tracking

↳ file is added to .ignore to prevent commiting to git.

③ **Store data Remotely**

dvc remote add myremote <remote-storage-URI>

④ **push the data:**

dvc push

⑤ **Track data changes with Git**

git add data.csv.dvc .gitignore

git commit -m "Added dataset"

⑤ Retrive Data.

to get the data back after cloning a project.

git clone <repo-url>

dvc pull (downloads the dataset from the remote storage)

⑥ Remove Data from Local storage.

dvc remove data.csv.dvc

→ stops tracking the file with Dvc

→ Does not delete the actual data unless you remove the file manually.

⑦ status:

dvc status

⑧ Reproduce ML pipelines

dvc repro

→ if you dataset, preprocessing script changes, this returns your ML pipeline.

⑨ Compare different Versions of data.

dvc diff (shows what changed in your dataset/s b/w different version)

⑩ dvc checkout

if git changed the branch, and you want to checkout that particular data.

MLflow Tracking Quickstart (from official documentation)

Step 1: Get MLflow (pip install mlflow)

↓

Step 2: Start a tracking server (mlflow.set_tracking_uri("http://<host>:<port>")
→ 127.0.0.1 : 5000 → local

↓

Step 3: Train a model and prepare Metadata for logging.

```
# Import mlflow
# from mlflow.models import infer_signature.

# Import ML libraries
# Load the dataset
# Train test split
# hyperparameter tuning
# Train the model
# predictions
# Calculating Metrics.
```

Best practise:
Dont train your ML model inside the mlflow run, if it has any errors, mlflow going to log few unnessasary logs (or) no logs

↓

Step 4: Log the model and its MetaData to MLflow.

```
# Set up tracking server.
# Create experiment
# Start MLflow run
    → log params
    → log metrics
    → set tag
    → signature
    → log the model
```

→

Step 5: Load the model as a python function (pyfunc) and use it for inference.

```
# load model back for predictions as generic python function model

loaded_model = mlflow.pyfunc.load_model(model_uri)
```

↓

Step 6: View the Run in the MLflow UI

```
# mlflow ui
```

Dagshub (Where we can
① Upload data, → using DVC
② Track your code, → using DVC, git
③ Log Experiments → using MLflow.)

Step 1: Create remote Repository using Dagshub (from Dagshub.com)

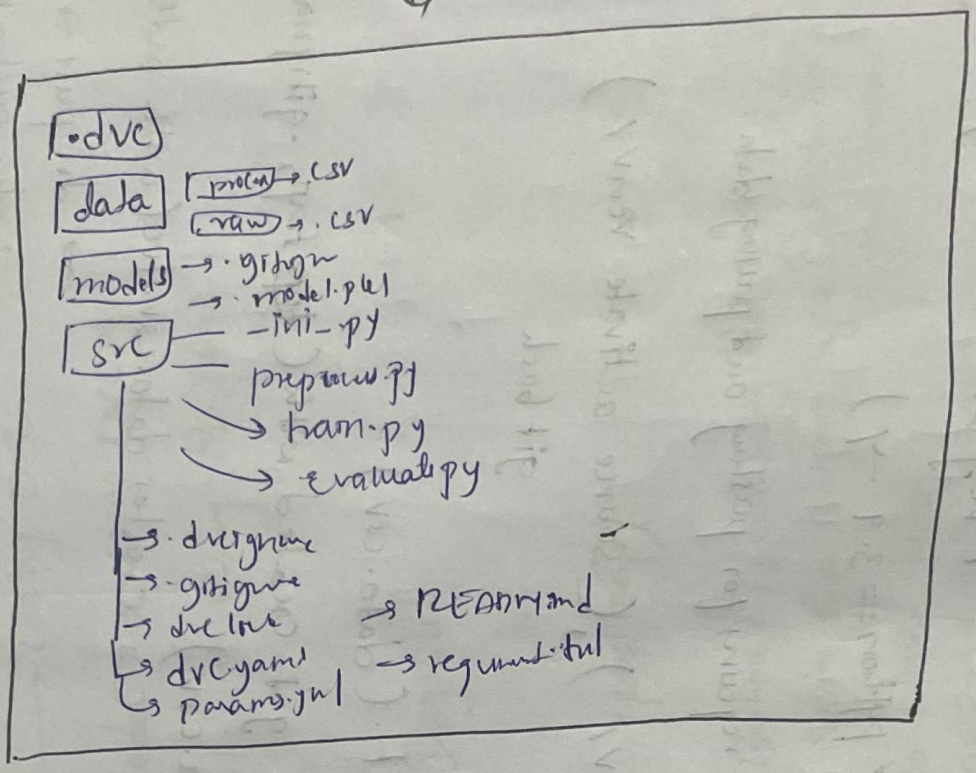
Step 2: Track your code (Initialize your git repository)

↳ git clone https://dagshub.com/Brahme27/demodagshub.git
↳ copies the exisiting repo (dagshub) from dagshub to your local computer
↳ cd demodagshub → Moves into cloned repo folder so you can start working inside it
↳ echo "# demodagshub" >> README.md
↳ git add README.md
↳ git commit -m "first commit"
↳ git branch -M main
↳ git push -u origin main
} common like git commands

Step 3: DVC with Dagshub Remote Repository / (or) similarly MLflow with dagshub remote Repository

↳ Create new environment (conda create -p venv python==3.9 -y)
↳ .gitignore (venv/)
↳ requirements.txt (dagshub, dvc, dvc-s3) ← requires for pushing and pulling data.
↳ activate new environment (conda activate venv/), (source activate venv/)
cmd gitbash
↳ pip install -r requirements.txt
↳ Create a folder (Data) and add dataset in it. (data.csv)
↳ initialize dvc (dvc init) → .dvc folder get created with (tmp, .gitignore, config files)
↳ add dataset to dvc (dvc add data\data.csv)
↳ Creates data.csv.dvc (with md5 file)
↳ git add data/data.csv
↳ git commit -m "first commit"
↳ to push data to dagshub (add a dagshub DVC remote, setup credentials) → git push -u origin main
→ dvc pull -r origin
→ dvc push -r origin

End-to-End ML Pipeline using Git, MLflow, Dvc, and Dagshub



Creating pipeline (preprocess → train → evaluate) using dvc stages

---

```
dvc stage add -n preprocess \
    -p preprocess.input, preprocess.output \
    -d src/preprocess.py -d data/raw/data.csv \
    -o data/processed/data.csv \
    python src/preprocess.py
```

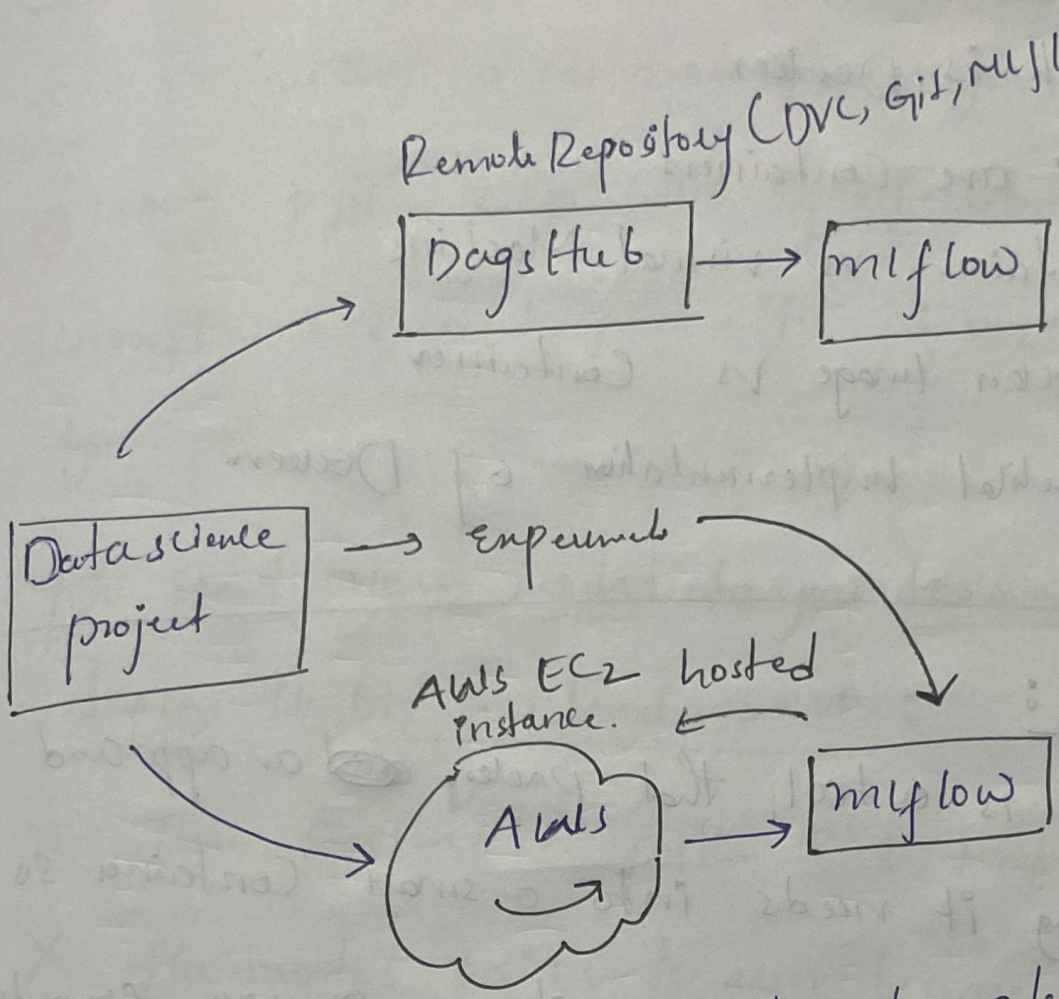
(preprocess) stage 1

↓

```
dvc stage add -n train \
    -p train.data, train.model, train.random_state, train.n_e
    -d src/train.py -d data/raw/data.csv \
    -o models/model.pkl \
    python src/train.py
```

(train) stage 2

MLflow with AWS CLOUD



for implementation Checkout project folder, clear steps are mentioned.

Evaluate

stage 3

→ dvc stage add -n evaluate \
-d src/evaluate.py -d models/model.pkl
python src/evaluate.py -d data/raw/data.csv \
-estimators, train.max_depth \

-p → params.yaml
-d → dependencies
-o → output

→ Run in git bash.

Dockers

1. What is Docker
2. What are Containers
3. Containers vs Virtual Machine
4. Docker Image vs Container
5. practical Implementation of Dockers

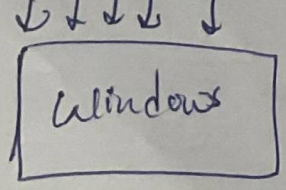
Docker:

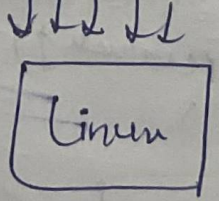
Docker is a tool that packs an app and everything it needs into a small Container so it runs the same anywhere — on your computer, a server or the Cloud.

Why Containers:

Data science appln

Anaconda MySql mongodb





Libraries
numpy
pandas

→ Dev envirnt

Dev A → Dev B

Real world Scenario without Containers:

you are building a ML model on your laptop using python 3.10 with specific libraries like tensorflow 2.9 and pandas 1.5 Everything works fine

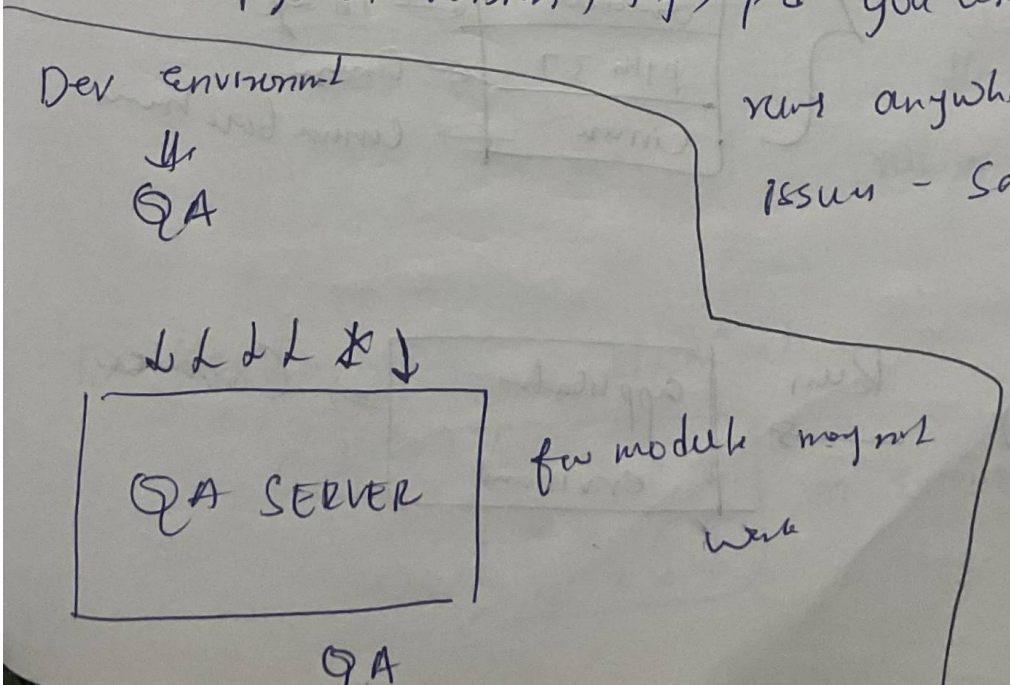
You send your model to your team member or deploy it on a cloud server:

✗ Their system has python 3.8, tensf 2.5, pan 1.3

✗ the model fails to run because of version mismatch.

Solution (Docker) ✓

with a container, you pack your model with the exact python version, tf, pd you used now, it runs anywhere without issues - Saves time and effort

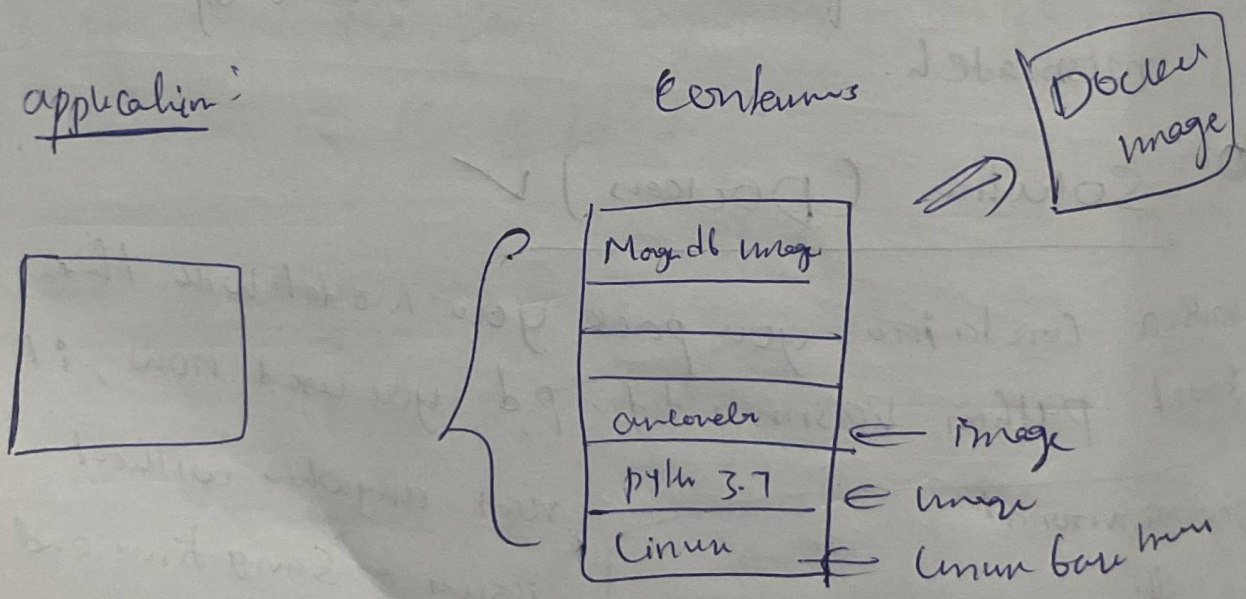


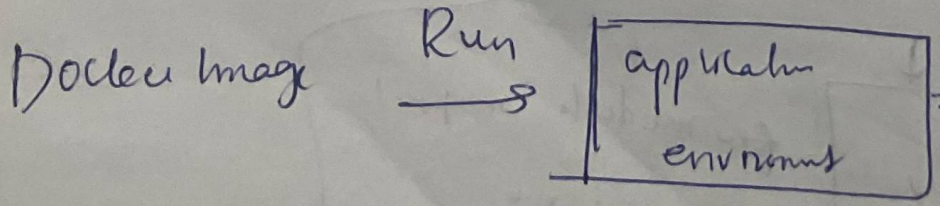
Containers:

① portable artifact
↳ easily share and move this package to any environment

② makes development and deployment more easy and efficient

Docker Image and Containers





Docker Image:

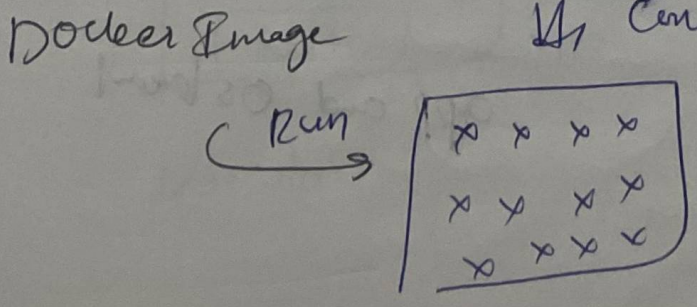
a pre built package that contains everything needed to run an application (code, dependencies, libraries)

Docker Container:

a running instance of a docker image that executes the application in an isolated environment

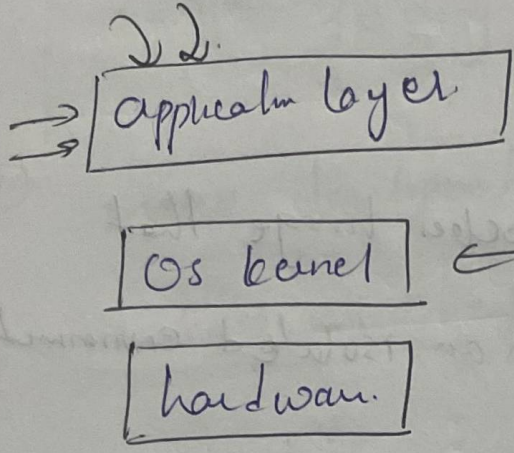
Difference

Docker Image	Container
① package or artifact	① Docker Image → Run
② Move or share this artifact	② Start the application
	③ Create a container (environment) running

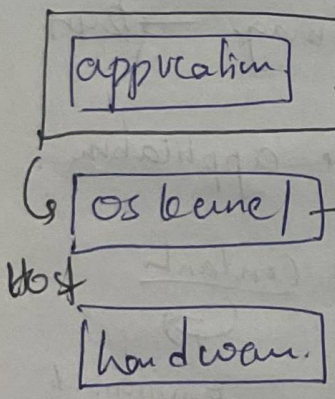


Docker vs Virtual Machines

Operating system:

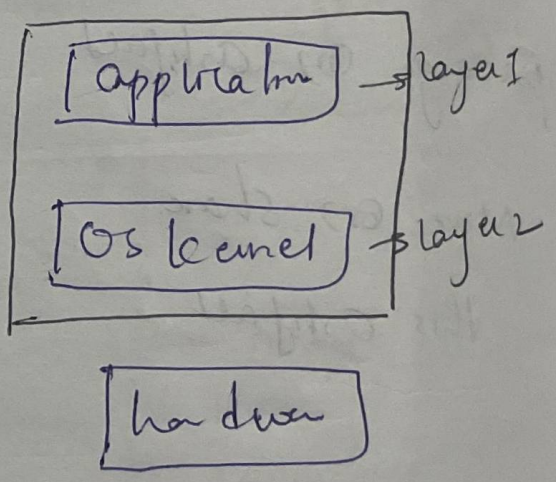


Docker



Docker virtualizes application layer.

Virtual machine



VM virtualizes both app and OS kernel

Key differences

Dockers

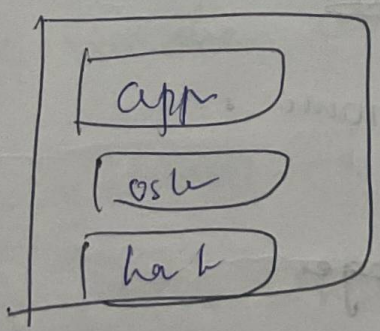
- Docker image size is usually small (MB)
  (bcoz On host machine it virtualizes on app layer)
- Dockers contains standard much faster

VM

- size will be huge (GB)
  (virtualizes both app and OS kernel)
- VM is comparatively slower.

Compatibility ★★★

- VM can be installed on any OS
- Docker image → compatibility issues.



Windows

Docker image of Linux

Older than windows 10

Docker practical implementation

app.py
requirements.txt
dockerfile

inside docker file

```
FROM python:3.8-alpine
WORKDIR /app
COPY . /app
RUN pip install -r requirements.txt
CMD python app.py
```

→ for building docker image

cs ~~docker build -t "welcome~~

docker build -t app-name .

→ for checking docker images

cs docker images

→ running docker images.

↳ docker run -p 5000:5000 brahma-app

↳ Number of containers are running

↳ docker ps

↳ Stop running docker container

↳ docker stop c7dfod0643b7

container id

Docker Basic Commands (Working with DockerHub)

→ pulling docker image from dockerhub

① docker pull "docker-image-name"

② docker images.

③ To run the docker image locally

docker run -d -p 80:80 docker/getting-started

④ To check whether container is running or not

docker ps

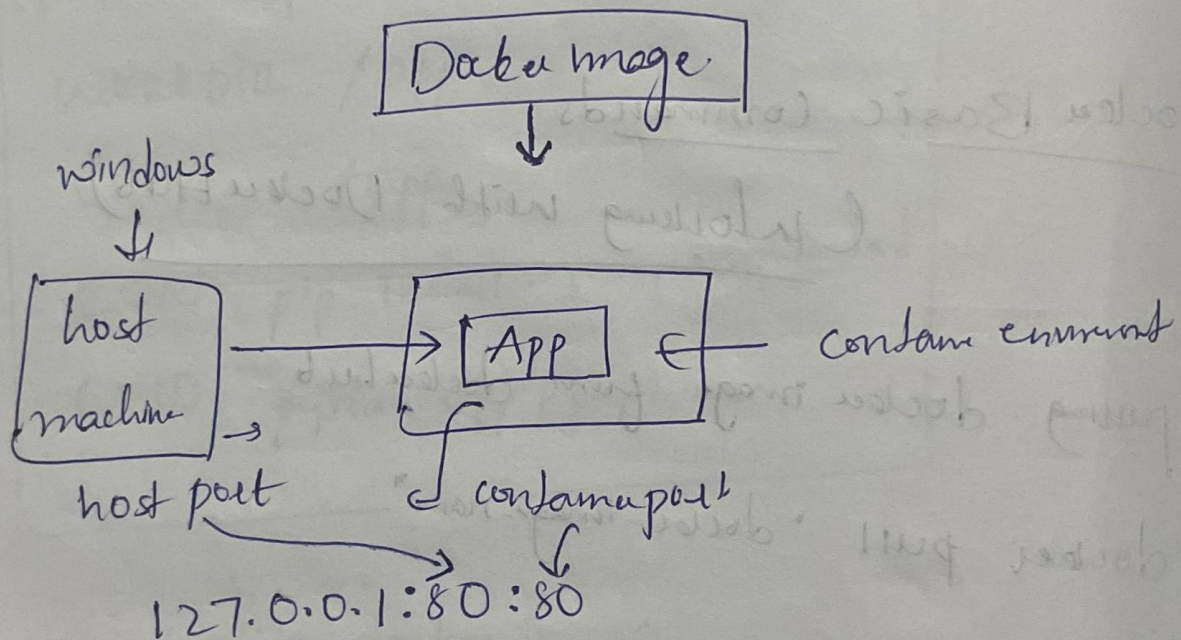
→ to stop the running container.

↳ docker stop container-id

→ to remove the docker image.

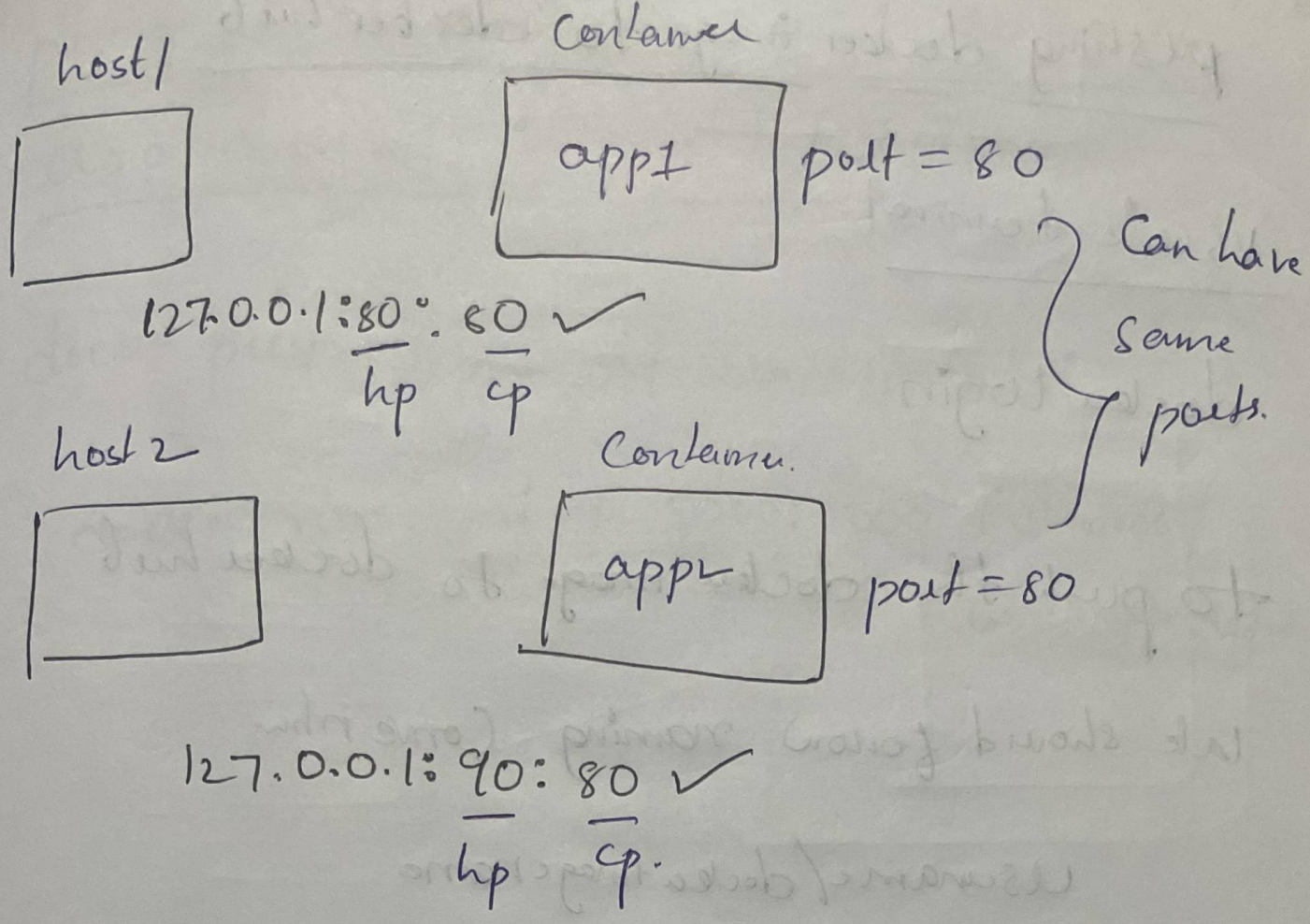
↳ docker image rm -f image-id

Container port and Host port



✡✡✡

We can run any number of containers with same container port but host port should be diff


host1
Container
app1
port = 80
127.0.0.1:80:80 ✓
hp cp
Can have
Same
ports.
host2
Container
app2
port = 80
127.0.0.1:90:80 ✓
hp cp.

pushing docker image to docker hub

VS code terminal

docker login

to push the docker image to docker hub

we should follow naming convention

Username/dockerimagename

→ Now remove the existing dimage or rename it

↳ docker rmi -f imageid

docker build -t brahme/welcome-app .

or

↳ docker tag krishnaik06/welcome-app krishnaik06/welcome-app1

pushing docker image to docker hub.

docker push krishnaik06/welcome-app:latest

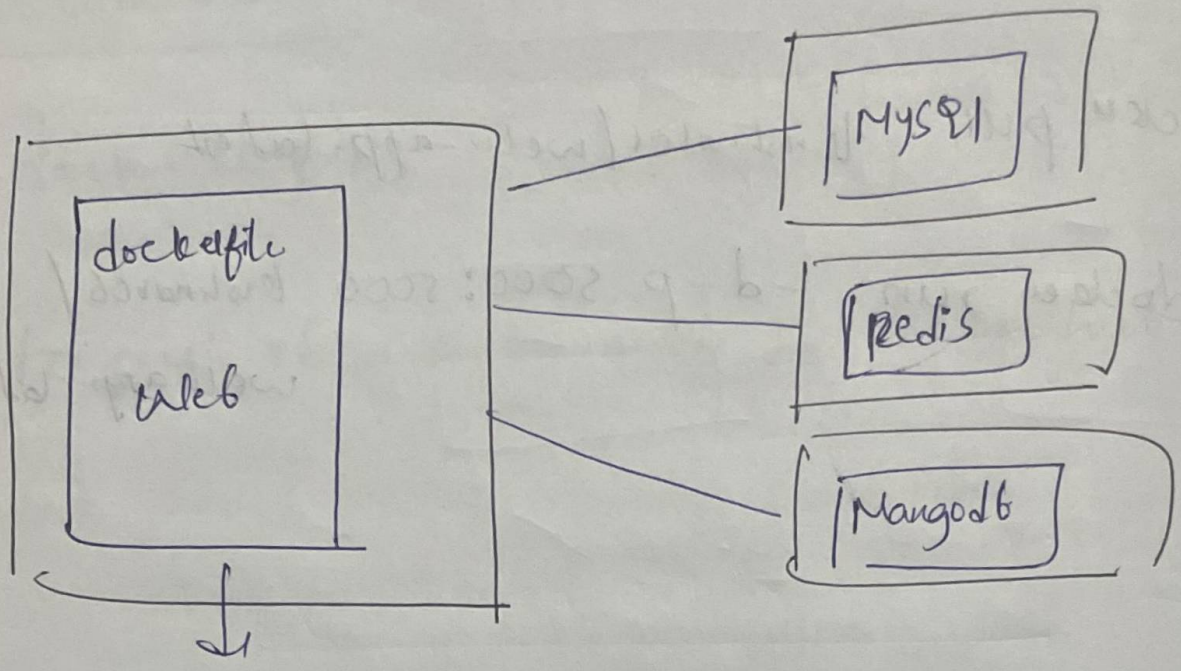
for dockerhub.

pulling docker-image back and running it as a container in my local

- docker pull lenshankar06/web-app:latest
- docker run -d -p 5000:5000 lenshankar06/webapp:latest

Docker compose

Generally, you are going to see different images for MySQL, Mongodb, ... in dockhub



Docker Compose: is a tool for defining and running multicontainer docker application

Docker compose.yaml

→ Create three images

→ run three at a time

docker compose up

→ docker compose up

→ docker compose stop.

Apache Airflow:

is an open source is an open source workflow automation and orchestration tool that helps schedule, monitor and manage complex data pipelines using python.

Use cases:

① **Data engineering and ETL:**

Automate data extraction, transformation, and loading (ETL) pipelines.

② **Machine learning:**

Schedule and manage ML model training, evaluation and deployment.

③ and many more.

Why apache airflow:

① No manual work, no need for engineer to run script daily

② handles dependencies. ensures step 2 happens only after step 1

③ Scalable

④ Error handling → work even if some of jobs happen every day

↳ Sends alerts if something goes wrong

Without Airflow: Engineer manually collect, clean and analyze data daily

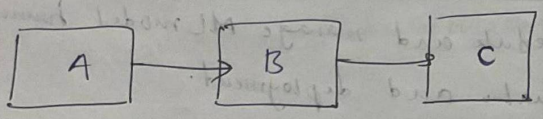
With Airflow: everything runs automatically at the right time, saving time and effort.

Key Concepts In Apache Airflow

① DAG (Directed acyclic Graph):

Collection of tasks that you want to schedule and run

Directed Graph



A >> B >> C

② Tasks:

tasks represents the individual unit of work in a DAG.

↳ task can be anything

1. python function
2. Querying a database
3. Sending an HTTP request

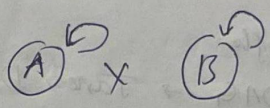
Core components of DAG

① Directed

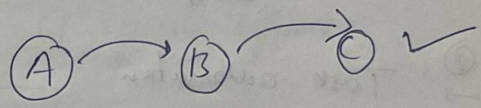
↳ Task must have a specific sequence

② Acyclic:

No task should depend on itself, it should depend on some other task.



Acyclic

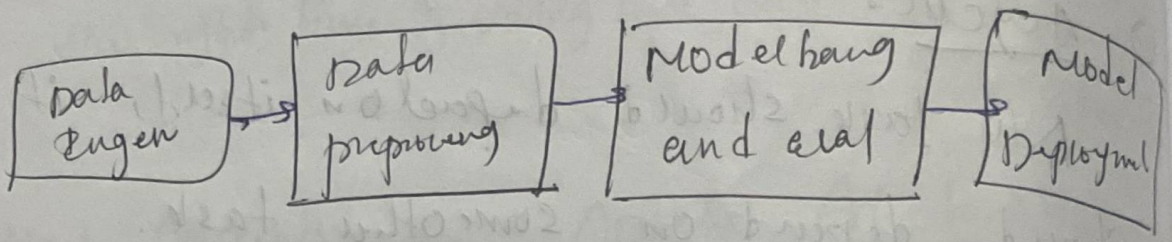


③ Dependencies:

Meaning that one task might need to finish before another task can start.

Why airflow for MLops

① → Orchestrating ML pipeline and ETL pipelines



apache airflow

DAG → Tasks, Dependencies

② → Task automation.

③ → Monitoring and alerts.

ETL pipeline → Weekly

Model Retraining → Every fortnight

→ Real time monitoring → airflow UI

→ task logs

→ Alerts and notifications → Emails

→ Retry mechanisms

↳ Retry task → pre defined

apache airflow: is a tool that helps you schedule, organize and run tasks automatically

Astronomer: makes it easy to use apache airflow. It helps you to install, manage and run airflow without much effort

Astronomer

→ Install astro CLI on your systems

→ `astro dev init`

used to initialize new astro project

→ Creates a new Astro project in the current dirctl.

→ Generate the required files and folder for running apache airflow locally.

→ Set up a Dockerfile, dags/, plugins/, requirements.txt to manage dependencies.

→ astro dev start

it starts all necessary airflow components (webserver, scheduler and worker) inside Docker Containers

→ it makes the airflow UI available on localhost

Make sure to run docker in background while your are working with airflow project

astro dev start

↳ to start the project

astro dev stop

↳ to stop the project

astro dev restart

↳ to restart the Containers

practical implementation of airflow using astro

- astro dev init
- inside dags/ folder create .py file.

Syntax for creating DAG

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
```

→ import libraries

```
define task 1
define task 2
define task 3
:
```

→ all define tasks

```
with DAG(.....) as dag:
    define tasks:
    task1 = PythonOperator(...)
    task2 = python operator(...)
    task3 = python operator( )
```

→ set up DAG

```
task1 >> task2 >> task3
```

→ set of dependency

ETL pipeline using Airflow

E - Extract
T - Transform
L - Load

ETL pipeline [Data pipeline]

Extract
Source

① API's
② Internal database
③ IoT Devices

Transform
Preprocessing
Transformation

→ JSON →

Load
Source

Data pipelines
Data engineer

① SQL db
② postgr db
③ Mongo db

using apache airflow we can easily perform ETL tasks

In our project

API's —Read→ Transformation → JSON → Database
postgresql

↓

Weekly, Day → Airflow
for scheduling tasks

ETL problem statement and project Structure

API

HTTP operator ← API
Connection → API
↳ airflow

↓

Transformation ← Generic python code

↓

Load → postgresql

Hooks: postgres hooks, S3 hooks

↳ Docker container

till now we created tasks using python operator

for creating task for API

↳ we use HTTP operator

✱✱✱✱

Create docker-compose file.

here by seeing the architecture we can say that here, docker, API's, airflow all are running in different containers, with docker compose we can interact with them.

Defining ETL DAG with implementing Steps: and

Import libraries

Define DAG.

Step1: Create the table if it does not exists

Step2: Extract the NASA API Data (APOD) ashonomy picture of the Day [Extract pipeline]

Step3: Transform the data (pick the information that we need to save)

Step4: Load the data into the database (postgres SQL)

Step5: Verify the data with DB Viewer

to check whether everything is working fine

Step6: Define the task dependencies

create_table() >> extract_apod

api_response = extract_apod.output } E

transformed_data = transform_apod_data(api_response) } T

load_data_into_postgres(transformed_data) } L

finally deploying into astronomer and AWS.

→ initialize (postgreshook) to interact with postgreSQL database.

→ Extract the data using (Simple HttpOperator)
used to make API calls (GET, post, etc.)
↳ stores response for later use.

→ perform some kind of transformation.

→ initialize (postgreshook) again to insert data into postgreSQL db.

→ Run the project astro dev start
→ Give two connections in airflow admin: http conn, api_conn.
→ Run the DAG → Verify your data in postgreSQL using DBviewer.

(also)

Deploy your project to astronomer cloud
Then you can run DAG by changing host to [Amazon RDS]
Run on astronomer cloud
↳ Stores extracted results into AWS (RDS) ↓ Postgres sql db. ↳ When we created postgre db

Github Actions

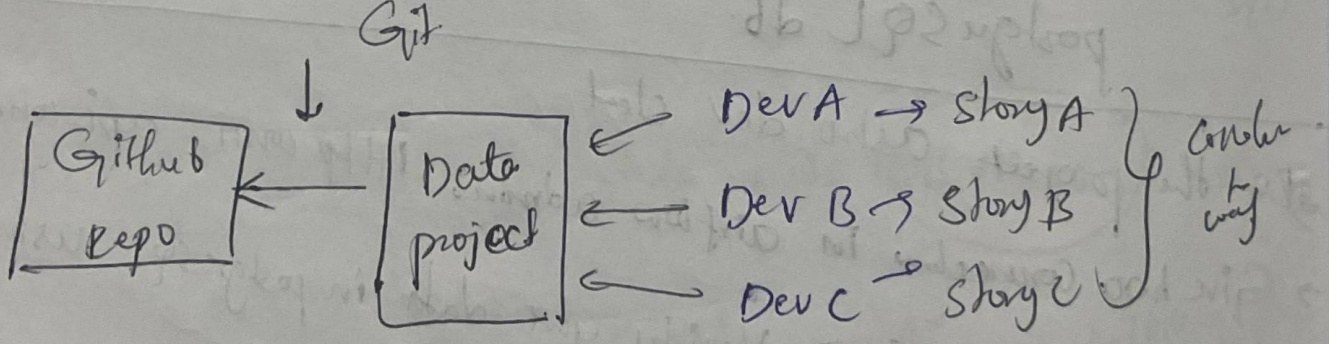
is an automation tool in Github that allows you to run workflows automatically based on events like code pushes, pull requests, or schedule tasks. It is mainly used for CI/CD (Continous Integration & Continous deployment) to automate testing, building and deploying code

key terms

Github → Code repository. → Collaborative env

↳ we commit our code over here

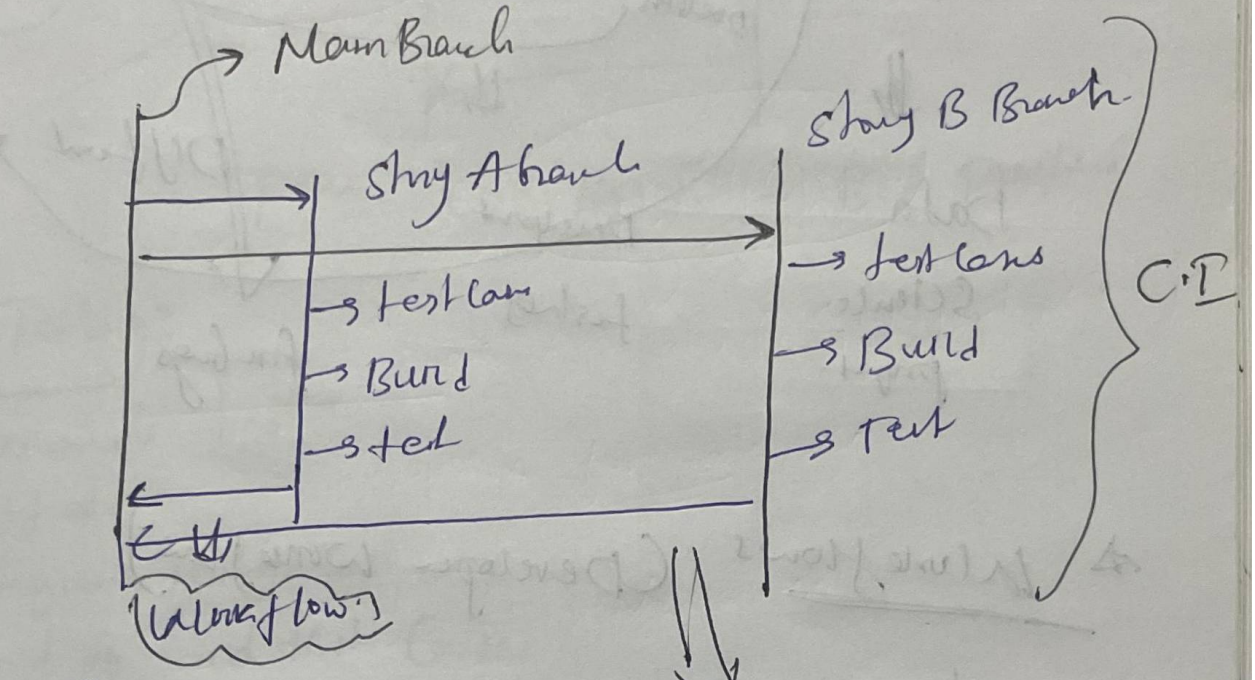
Git → Distributed Version Control System (Devops)



Github actions: (CI/CD tool)

Data science project

1. Story A
2. Story B
3. Story C



1. test Cases
2. Build proper

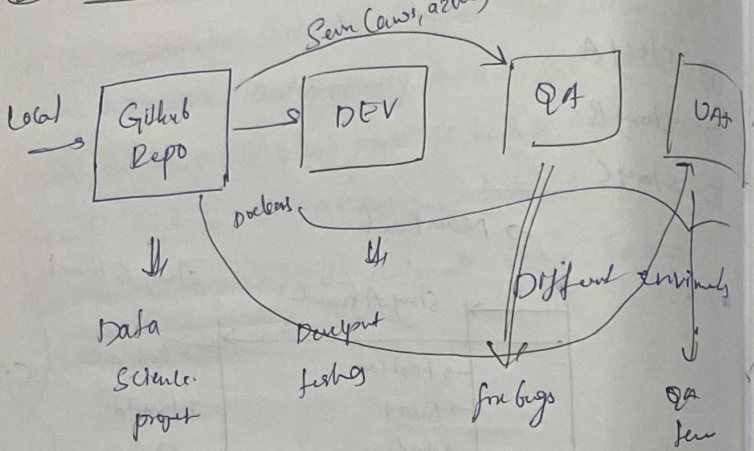
Conflict

Resolve Conflict

Workflow

1. Test Cases
2. Review the Code
3. Building proper

② Continuous Deployment (CD)



⚠ Workflows (Developer Workflow):

key stages

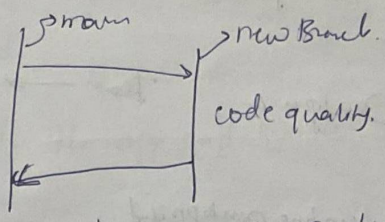
① Coding → IDE (VS code) → python, js, ... →

② Version Control: Git → manage the codebase, multiple

↳ Commit
↳ Branches
→ push
→ pull
→ Resolve Conflict

live website.

⑧ Code Review:



Peer Code Review → Best practices.

④ Testing:

automate testing

① Unit testing
② Integration testing
③ end to end test cases

coding standards, Best practise.

developer to collaborate

⑤ Continuous Integration (CI)

Building, testing → Modified → Running the same

⑥ Continuous Deployment (CD):

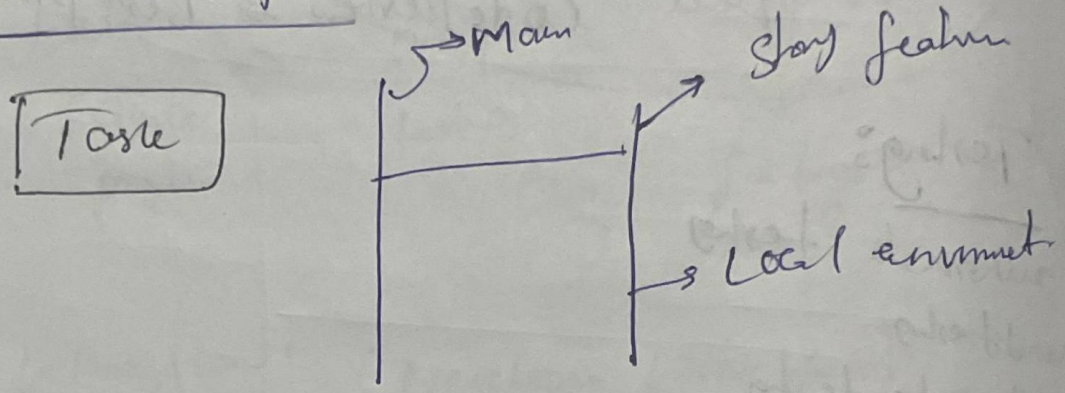
Deploy in diffrent Server (Dev, QA, UAT, Prod)

Developer Workflow with Real world

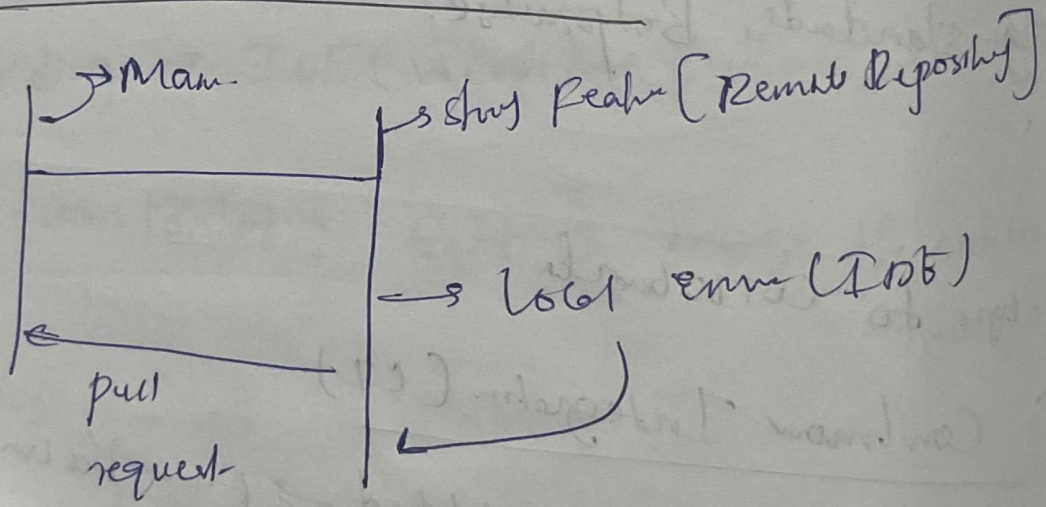
Example:

Developer A → Team → Data science project

① Feature Development



② push and pull requests (PR)



Other team member review the PR for code quality, style and any other issues

demo making

③ Automated CI pipeline

① once the pull req is opened, A CI pipeline is automatically triggered

② This pipeline builds the app and runs all Test Cases

③ If pipeline passes, the PR is approved and merged into main branch

④ Continuous Deployment

① upon merging the PR, a CD pipeline is triggered

② The application is automatically deployed to Dev env or Stage env

③ next deployed to prod

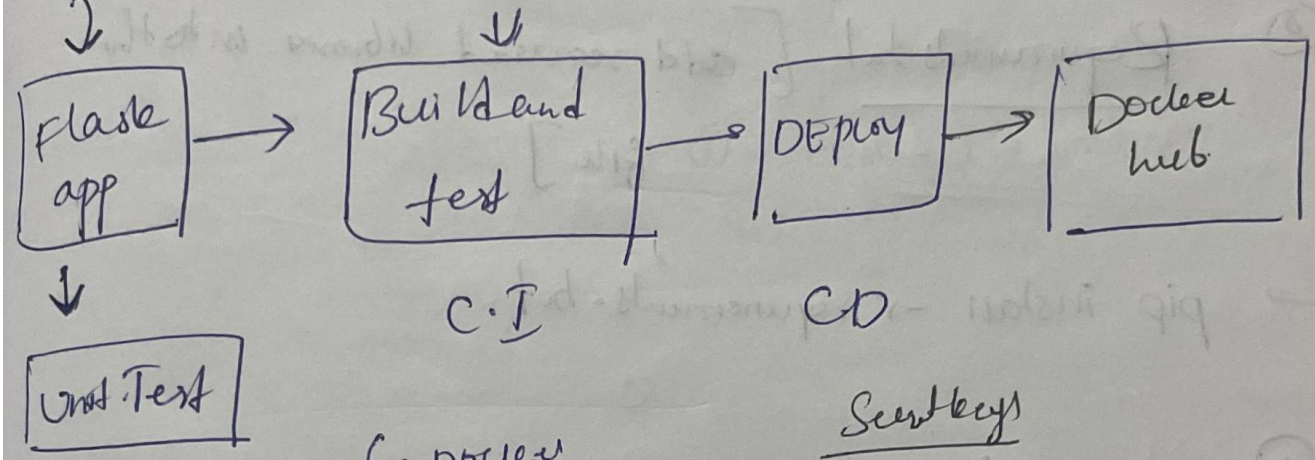
Benefits of Developer Workflow:

1. Improved Collaboration:
2. Higher Code Quality
3. Reduced Errors
4. Faster Delivery
5. Continuous Feedback Loop.

End to End Github Action Workflow project with Dockerhub

Github action Workflow

- CI-CD pipeline
- Git action workflow



- Docker image

Secret keys

1. Docker username
2. Docker password (Token)

tools we use

- Git
- Github
- Docker
- pytest
- Flask

End-to-End Datascience project

① Step up github (Cloning the Repository)

(.gitignore, LICENSE, README.md)

② Creating an Environment

→ Conda create -p venv python==3.10 -y

③ Requirements.txt [add required libraries on to this file]

→ pip install -r requirements.txt

④ template.py

Write some genric structure for your project

⑤ logger implementation

→ Implying in src.__init__ for logging information

⑥ Add some common function in Utils

↳ Config Box.

↳ @ ensure_annotations

① Building ML pipeline

1. Data Ingestion
2. Data Transformation
3. Model trainer
4. Model evaluation
5.

⇒

Workflows

1. Update config.yaml
2. Update schema.yaml
3. Update params.yaml

(Update for each and every ML pipeline)

4. Update the entity
6. Update the configuration manager in src config
6. Update the components
7. Update the pipeline
8. Update the main.py